

## Lab 1: Grade Evaluator & Archiver



| SUBJECT | 79 | 82 | 75 | 62 | 61 | 68 | 67 | 81 | 89 | 80 | 87 | 68 | 69 | 68 | C |
|---------|----|----|----|----|----|----|----|----|----|----|----|----|----|----|---|
|         |    |    |    |    |    |    |    |    |    |    |    |    |    |    |   |

### Overview

You are building a Python application that calculates a student's final academic standing based on a pre-existing CSV file of course grades. You will then build a Bash shell script that acts as an "organizer" to find your CSV files. We have provided a starter Python template that handles the file reading for you.

---

**1. The Data Source:** The csv data for the grades are located in grades.csv.

### 2. Python Application Features (Your Tasks)

Using the provided template, you must implement the following features in your Python script:

- **Grade Validation:** Check that the score for every assignment respects the assignment grade range (between 0 and 100).
- **Weight Validation:** Verify that the sum of all assignment weights equals exactly 100. Ensure all "Summative" assignments add up to exactly 40, and all "Formative" assignments add up to exactly 60.
- **GPA Calculation:** Calculate the final GPA based on the weighted scores. Use the formula:  $GPA = (Total\ Grade / 100) * 5.0$ .
- **Final Decision (Pass/Fail):** A student passes only if they score at or above 50% in **both** categories. Print the final status of the student (**PASSED** or **FAILED**).
- **Resubmission Logic:** Resubmission is for failed formative assignment that carries the **highest weight**. If multiple failed formative assignments share the same highest weight, the program must display all of them as eligible for resubmission.
- **TESTING:** Cross-check your GPA calculations, status, and assignment to resubmit with the [provided transcript](#) structure to ensure accuracy

### 3. Shell Script Organizer (`organizer.sh`)

Write a Bash shell script named `organizer.sh` that fully automates the archiving of your course data and logs its actions.

- **Archive Directory:** Check if a directory named `archive` exists in the current location. If it does not, create it.
- **Timestamp Generation:** Create a timestamp string representing the current date and time.
- **The Archival Process:** \* Rename the existing `grades.csv` file by appending the generated timestamp to the filename (e.g., `grades_20251105-170000.csv`). Move the original file to the archive directory with its new, timestamped name.
- **Workspace Reset:** Immediately after moving the file, create a new, empty file named `grades.csv` in the current directory so the environment is ready for the next batch of grades.
- **Logging:** Append the archiving details of this operation (timestamp, original filename, and the new archived filename) into a log file named `organizer.log`. The log file should accumulate entries from every run.

### 4. Python Starter Template (`grade-evaluator.py`)

Use this starter code to begin your Python application. It handles the CSV parsing so you can focus on the logic. *The file containing the source code is attached to this assignment below.*

*(You are free to change the code accordingly)*

### 5. Evaluation Criteria

When building your solution, cross-check your implementation against these key details:

- **Robust Error Handling:** Does your program gracefully handle a missing `grades.csv` file? What happens if the CSV is completely empty (as created by your shell script)?
- **Formula Accuracy:** Cross-check your GPA calculations, status, and assignments to resubmit to ensure accuracy. Remember that passing requires a 50% minimum in **both** the Formative and Summative groups, not just a 50% total overall.
- **Correct Weight Tracking:** Ensure your program strictly enforces the 60/40 split.

# INDIVIDUAL CODING LAB

- **Archival & Logging Implementation:** Correct implementation of the archival process and logging. Verify that your shell script successfully creates the archive folder, moves/renames the file, leaves behind a fresh CSV, and writes to **organizer.log**.

## 6. SUBMISSION INSTRUCTIONS

Submit a Github repository link containing your project files. Your repository must include exactly:

1. **grade-evaluator.py**: Your completed Python script.
2. **organizer.sh**: Your completed Bash script.
3. **Readme.md**: A markdown file containing clear usage instructions on how to run both your Python application and your shell script.

**Github repository name format:** **{LabNbr}\_{Github\_username}**, **e.g:** lab1\_ndinelao1

---

## 7. Learning Objectives

By completing this assignment, you will be able to:

1. Apply file handling techniques to read, parse, and process structured data from text files.
2. Use conditional logic to validate input, ensure data integrity, and determine complex pass/fail statuses.
3. Automate system tasks using shell scripts to organize files and generate persistent logs.

## Lab 2: The Social Media Data Detective



### Overview

You have just been hired as a Junior Data Analyst for a digital marketing agency. Your first quest is to analyze a massive, messy dataset of social media posts downloaded from Twitter.

Since you are evaluating core programming logic, your manager has imposed a strict rule: **You are strictly prohibited from using Python's built-in `.sort()`, `sorted()`, or `max()` functions.** You must write your own custom algorithms to sort and search the data. You will also use Bash shell scripting to see how the command line can sometimes do a Python script's job in just one line of code!

### 1. The Data Source (CSV Structure)

Your program will read a large CSV file named [twitter\\_dataset.csv](#). Login/signup on Kaggle to be able to download the csv file. Download only the "twitter\_dataset.csv" file. The file follows this exact format:

### 2. Python Application Features (Your Tasks)

Using the provided starter template, you must implement the following features. Remember to use list properties like `len()` to check sizes and `.append()` to build new lists!

- **Quest 1: The Data Auditor (Handle Missing Fields)**

Before you can analyze the data, you must clean it. Write a function that loops through your list of tweets. If a tweet is missing its **Text**, remove it from the list (or ignore it). If a tweet is missing its **Likes** or **Retweets**, replace the empty blank with a **0**. Print out how many "bad" rows were fixed or removed.

- **Quest 2: The Viral Post (Find the Maximum)**

He wants to know which post got the most engagement. Write a function that loops through your dataset to find the single tweet with the absolute highest number of **Likes**. Print the **Username**, the **Likes**, and the **Text** of that viral tweet.

# INDIVIDUAL CODING LAB

## INTRODUCTION TO PYTHON PROGRAMMING AND DATABASES

### BSE YEAR 1 TRIMESTER 2

- **Quest 3: The Algorithm Builder (Custom Sort)**

The marketing team wants to see the "Top 10 Most Liked Tweets." Write your own sorting algorithm (like **Bubble Sort** or **Selection Sort**) to order the list of tweets from highest **Likes** to lowest. Do not use `.sort()`. Once sorted, slice the list to print only the top 10.

- **Quest 4: The Content Filter (Search & Extract)**

Write a function that asks the user for a search word (e.g., "Python"). Iterate through your dataset and extract any tweet that contains that word into a *brand new list*. Print the `len()` of this new list to tell the user how many tweets matched their search, followed by the tweets themselves.

### 3. Shell Script Analyzer (**feed-analyzer.sh**)

Now, let's do Quest 2 again, but using Bash! Write a Bash script named **feed-analyzer.sh** that takes **twitter\_dataset.csv** as an input and does the following:

- Extract just the **Username** column.
- Sort the usernames, count how many times each one appears, and sort them again by the highest count.
- Display the **Top 5 Most Active Users**.
- *Hint:* Research how to chain together terminal commands like `cut -d',' -f2`, `sort`, `uniq -c`, and `head` using the pipe `|` operator.

### 4. Python Starter Template (**data-detective.py**)

Use this starter code. It handles opening the CSV file so you can focus entirely on the logic quests. *The file containing the source code is attached to this assignment below.*

(You are free to change the code accordingly)

### 5. Evaluation Criteria

When building your solution, cross-check your implementation against these key details:

- **List Mastery:** Are you correctly using `len()` to check array sizes? Are you correctly appending items to new lists during your keyword search? Are you properly tracking variables during loops (like finding the maximum value)?
- **No Built-in Sorting or Max:** The use of `.sort()`, `sorted()`, or `max()` will result in a 0 for those respective quests. You must write the logical loops yourself.

# INDIVIDUAL CODING LAB

- **Data Types:** When comparing **Likes**, remember that CSV files read everything as text strings (e.g., **"150"**). You must convert them to integers using `int()` before comparing them, otherwise **"90"** will incorrectly be evaluated as larger than **"100"**.
- **Shell Script Accuracy:** Your `feed-analyzer.sh` must be a single pipeline or a few short commands that cleanly print the top 5 users and their counts.

## 6. SUBMISSION INSTRUCTIONS

Submit a Github repository link containing your project files. Your repository must include exactly:

1. `data-detective.py`: Your completed Python script.
2. `feed-analyzer.sh`: Your completed Bash script.
3. `twitter_dataset.csv`: The dataset file used for your testing.
4. `Readme.md`: A markdown file containing usage instructions, and a brief 2-sentence explanation of *how* your custom sorting algorithm works.

Github repository name format: `{LabMbr}_{Github_username}`, e.g: `lab2_hmusangwa`

---

## 7. Learning Objectives

By completing this assignment, you will be able to:

1. Clean and process real-world datasets by identifying and handling missing or empty fields.
2. Traverse and manipulate Python lists and dictionaries to count frequencies and extract specific data points.
3. Implement fundamental sorting algorithms (like Bubble or Selection Sort) from scratch to understand algorithmic logic.
4. Utilize Linux command-line text processing tools to achieve complex data analysis with minimal code.